

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 12. Sets

Lecture: 133. Set Comprehensions

Section 1: Lecture Summary

This lecture covers **set comprehensions** in Python, a concise way to create sets using a similar syntax to list comprehensions but with curly braces. It highlights how to construct a set comprehension by combining an expression with an iterable, using a `for` loop inside curly braces. The lecture explains that unlike lists or tuples, sets automatically discard duplicate values, so the resulting set contains only unique elements. Several examples demonstrate generating sets from ranges, lists with negative and positive integers (including squaring for values), and from strings with letter case normalization. The instructor emphasizes the similarity to list comprehensions, the difference being bracket type and the uniqueness property of sets. Finally, the lecture encourages practice and exploring additional features like conditional expression inside set comprehensions.

Section 2: Key Concepts and Explanations

- Definition of Set Comprehension

- A Python set comprehension is a compact way to create a set by combining an expression and a loop inside curly braces `{}`.

- Syntax:

```
{ expression for item in iterable }
```

- It closely resembles list comprehension but uses curly braces instead of square brackets.

- Characteristics of Sets in Comprehension

- Sets contain **only unique elements** – duplicates are automatically removed.
- Order is not guaranteed; Python's interactive interpreters like Jupyter may display sets in sorted order, but sets themselves are unordered collections.

- **Step-by-step Construction**

- Start with the iterable (e.g., `range(1,5)`).
- Use a `for` clause to iterate through elements.
- Use an expression (often just the loop variable or a transformation) to specify the values in the set.

- **Difference from List and Tuple Comprehensions**

- Brackets: Curly braces `{}` vs. square brackets `[]` for lists and parentheses `()` for tuples.
- No commas or stars `*` needed, just braces and the comprehension syntax.
- Automatic removal of duplicates when using sets.

- **Examples of Set Comprehension Use**

- Constructing a set from a range of integers (1 to 4).
- Creating a set of squares from a list of both negative and positive integers, showing that duplicates from symmetric squares are removed.
- Generating a set of lowercase letters from a string, demonstrating case normalization with `.lower()` and duplicate letter elimination.

- **Additional Notes**

- You can combine set comprehensions with conditional expressions and functions for more complex set creation.
- Seeing the real insertion order requires iteration (e.g., using a `for` loop to print elements) since Jupyter shows sorted sets.
- Set comprehension is a versatile method for constructing sets dynamically in one line, ideal for filtering and transforming data.

Section 3: Example Code and Use Cases

Example 1: Simple set comprehension from a range

```
S1 = {x for x in range(1, 5)}  
print(S1)  
print(type(S1))
```

Explanation: Creates a set of integers from 1 to 4 using range and set comprehension. Prints the set and confirms its type as `set`.

Example 2: Set comprehension with squares of numbers including negatives

```
S2 = {x**2 for x in [-2, -1, 0, 1, 2]}  
print(S2)
```

Explanation: Squares each value in the list and constructs a set of unique squares. Demonstrates duplicate values (4, 1) only appear once.

Example 3: Set comprehension from a string with lowercase conversion

```
S3 = {x.lower() for x in 'PhilippInes'}  
print(S3)
```

Explanation: Iterates over each character in the string, converts it to lowercase, and adds it to the set, eliminating duplicates and showing distinct letters only.

Section 4: Key Takeaways

- Set comprehension uses curly braces `{}` with an expression and a `for` clause to generate sets in one line.

- Sets automatically **remove duplicate elements**; only unique items appear in the resulting set.
- The syntax is very similar to list comprehension, the only difference being the use of `{}` instead of `[]` or `()`.
- Set comprehensions can be applied to any iterable, including ranges, lists, or strings.
- You can transform the input elements (e.g., squaring numbers, converting to lowercase) within the comprehension expression.
- While Python sets are unordered, outputs in some environments like Jupyter may appear sorted.
- Set comprehensions support more complex expressions, including conditionals and function calls, enabling versatile set construction in Python.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



Set Comprehensions

Set Comprehensions

$S = \{ \}$

Set Comprehensions

$S = \{ \text{iterable} \}$

Set Comprehensions

$S = \{ \text{for iterable} \}$

Set Comprehensions

```
S = { for item iterable }
```

Set Comprehensions

```
S = { for item in iterable }
```


Set Comprehensions

```
S = {exp for item in iterable }
```

Set Comprehensions

```
S = {exp for item in iterable }
```

```
for x
```

Set Comprehensions

```
S = {exp for item in iterable }
```

```
for x in range(1,5):
```

Set Comprehensions

```
S = {exp for item in iterable }
```

```
for x in range(1,5):  
    print(x)
```

Set Comprehensions

```
S = {exp for item in iterable}
```

```
print(x) for x in range(1,5):
```

Set Comprehensions

```
S = { exp for item in iterable }
```

```
x for x in range(1,5):
```

Set Comprehensions

```
S = { exp for item in iterable }
```

```
{ x for x in range(1,5) }
```

Set Comprehensions

```
S = {exp for item in iterable}
```

```
S1 = {x for x in range(1,5)}
```

Set Comprehensions

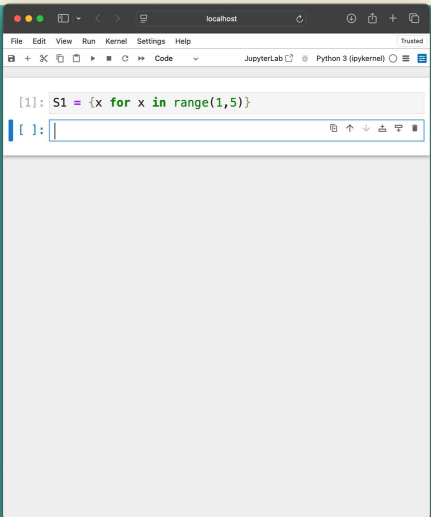
```
S = {exp for item in iterable}
```

```
S1 = {x for x in range(1,5)}
```

Set Comprehensions

$S = \{ \text{exp for item in iterable} \}$

$S1 = \{ x \text{ for } x \text{ in range}(1,5) \}$

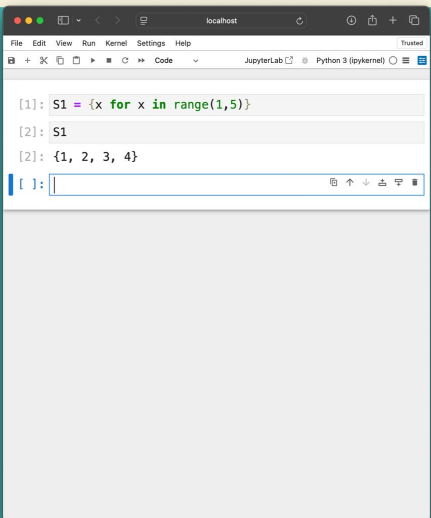


The image shows a JupyterLab window with a single code cell. The code in the cell is `[1]: S1 = {x for x in range(1,5)}`. The cell is currently empty, and the output area is also empty.

Set Comprehensions

$S = \{ \text{exp for item in iterable} \}$

$S1 = \{ x \text{ for } x \text{ in range}(1,5) \}$

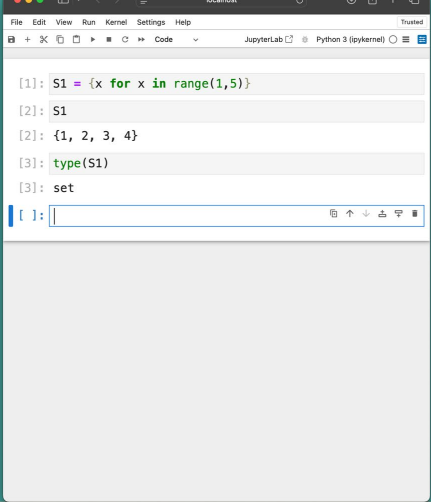


The image shows a JupyterLab window with three code cells. The first cell contains `[1]: S1 = {x for x in range(1,5)}`. The second cell contains `[2]: S1`. The third cell contains `[2]: {1, 2, 3, 4}`. The output area is empty.

Set Comprehensions

```
S = {exp for item in iterable}
```

```
S1 = {x for x in range(1,5)}
```

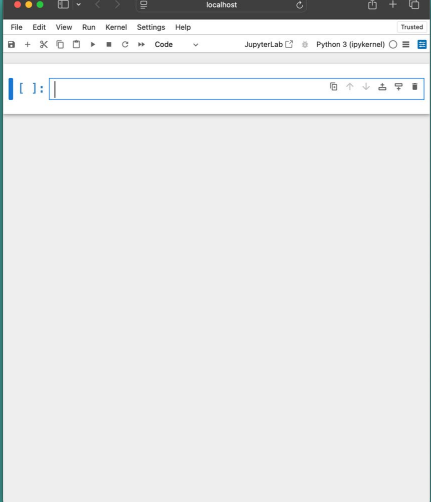


A screenshot of a JupyterLab interface. The code cell contains four lines: `[1]: S1 = {x for x in range(1,5)}`, `[2]: S1`, `[2]: {1, 2, 3, 4}`, `[3]: type(S1)`, and `[3]: set`. The output area is empty.

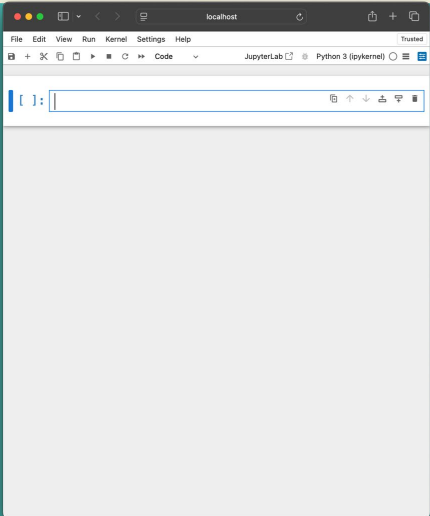
Set Comprehensions

```
S = {exp for item in iterable}
```

```
S1 = {x for x in range(1,5)}
```



A screenshot of a JupyterLab interface. The code cell contains two lines: `[ ]:` and `[ ]:` . The output area is empty.



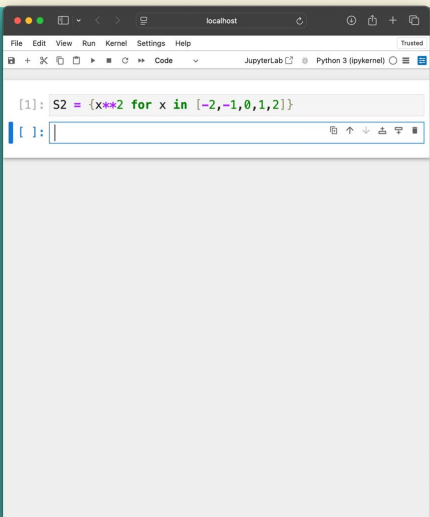
A screenshot of a JupyterLab interface on a localhost. The top bar shows 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. Below the bar, the 'Code' tab is active, showing an empty code cell with a prompt '[]: |'.

Set Comprehensions

$S = \{ \text{exp for item in iterable} \}$

$S1 = \{ x \text{ for } x \text{ in range}(1,5) \}$

$S2 = \{ x**2 \text{ for } x \text{ in } [-2,-1,0,1,2] \}$



A screenshot of a JupyterLab interface on a localhost. The top bar shows 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. Below the bar, the 'Code' tab is active, showing a code cell with the following code:

```
[1]: S2 = {x**2 for x in [-2,-1,0,1,2]}
```

The code cell is followed by a prompt '[]: |'.

Set Comprehensions

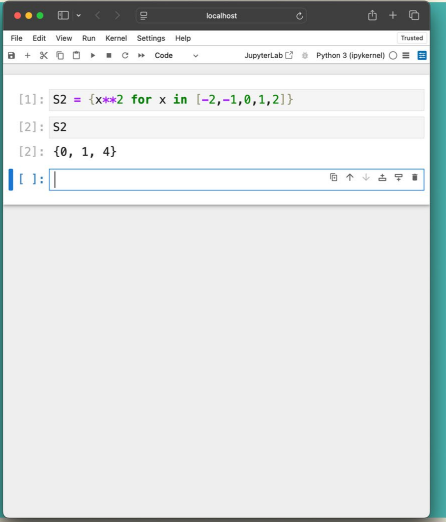
$S = \{ \text{exp for item in iterable} \}$

$S1 = \{ x \text{ for } x \text{ in range}(1,5) \}$

$S2 = \{ x**2 \text{ for } x \text{ in } [-2,-1,0,1,2] \}$

Set Comprehensions

$$S = \{ \text{exp for item in iterable} \}$$
$$S1 = \{ x \text{ for } x \text{ in range}(1,5) \}$$
$$S2 = \{ x**2 \text{ for } x \text{ in } [-2, -1, 0, 1, 2] \}$$



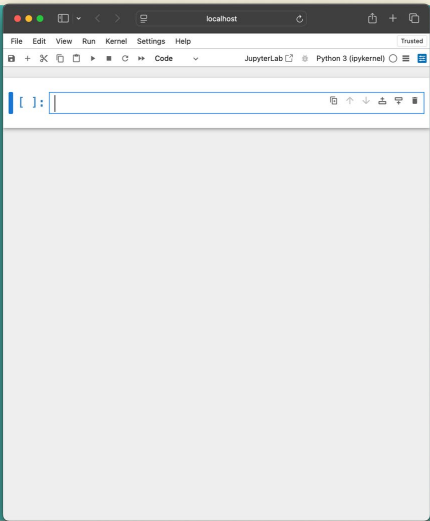
The screenshot shows a JupyterLab notebook interface with the following content:

```
[1]: S2 = {x**2 for x in [-2,-1,0,1,2]}
```

[2]: S2

[2]: {0, 1, 4}

[]:

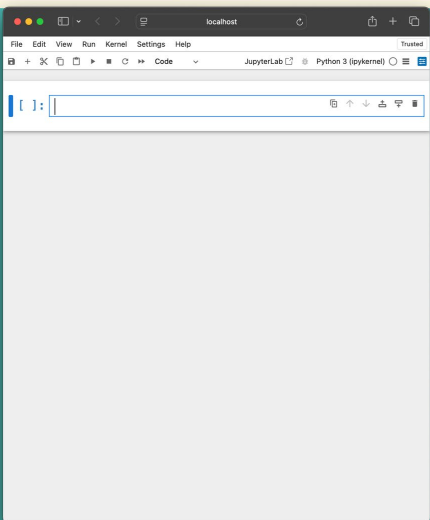


Set Comprehensions

$S = \{ \text{exp for item in iterable} \}$

$S1 = \{ x \text{ for } x \text{ in range}(1,5) \}$

$S2 = \{ x**2 \text{ for } x \text{ in } [-2, -1, 0, 1, 2] \}$



Set Comprehensions

$S = \{ \text{exp for item in iterable} \}$

$S1 = \{ x \text{ for } x \text{ in range}(1,5) \}$

$S2 = \{ x**2 \text{ for } x \text{ in } [-2, -1, 0, 1, 2] \}$

$S3 = \{ x.lower() \text{ for } x \text{ in 'PhilippInes'} \}$

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: S3 = {x.lower() for x in 'PhilippInes'}
[]:

Set Comprehensions

S = {exp for item in iterable}

S1 = {x for x in range(1,5)}

S2 = {x**2 for x in [-2,-1,0,1,2]}

S3 = {x.lower() for x in 'PhilippInes'}

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: S3 = {x.lower() for x in 'PhilippInes'}
[2]: S3
[2]: {'e', 'h', 'i', 'l', 'n', 'p', 's'}
[]:

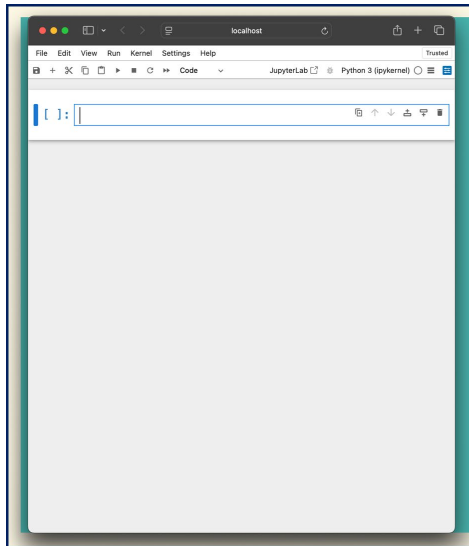
Set Comprehensions

S = {exp for item in iterable}

S1 = {x for x in range(1,5)}

S2 = {x**2 for x in [-2,-1,0,1,2]}

S3 = {x.lower() for x in 'PhilippInes'}

A screenshot of a JupyterLab web interface. The browser address bar shows 'localhost'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The main area is a code editor with a Python prompt '[]:' and a console area below it.

Set Comprehensions

$$S = \{ \text{exp for item in iterable} \}$$

$$S1 = \{ x \text{ for } x \text{ in range}(1,5) \}$$

$$S2 = \{ x^{**2} \text{ for } x \text{ in } [-2,-1,0,1,2] \}$$

$$S3 = \{ x.lower() \text{ for } x \text{ in 'PhilippInes'} \}$$